

Informe de Implementación de Autenticación y Autorización con JWT y Control de Acceso Basado en Roles (RBAC)

Descripción General del Sistema

Se ha desarrollado un sistema de autenticación y autorización para una aplicación web orientada a la gestión de empleados. El sistema permite autenticar usuarios mediante tokens JWT y restringir el acceso a los recursos internos en función de los roles asignados, siguiendo el modelo RBAC. Las operaciones disponibles están diferenciadas según el tipo de usuario: Empleado, Gerente o Administrador.

El sistema ha sido diseñado con separación clara entre autenticación y autorización, empleando buenas prácticas de seguridad, persistencia de datos y arquitectura modular.

Arquitectura Técnica

- **Backend:** Node.js con Express.js
- **Base de datos:** MongoDB (modelo relacional flexible para usuarios y roles)
- **Librerías utilizadas:**
 - `jsonwebtoken` para generación y verificación de tokens JWT.
 - `bcrypt` para hashing seguro de contraseñas.
 - `dotenv`, `express-validator`, y middleware personalizado para verificación de tokens y permisos.
- **Formato del token JWT:**
 - Algoritmo: HS256
 - Claims incluidos: `userId`, `role`, `exp`

Modelo de Usuarios

Cada usuario en la base de datos contiene los siguientes campos relevantes:

- `username`: Identificador único para login.
- `password`: Contraseña hasheada.

- `role`: Enumeración con valores posibles: `EMPLOYEE`, `MANAGER`, `ADMIN`.

La creación de usuarios puede realizarse manualmente desde un seed inicial o desde un endpoint de registro restringido.

Proceso de Autenticación

1. El usuario ingresa sus credenciales en el endpoint de login.
2. El sistema compara el hash de la contraseña ingresada con el hash almacenado.
3. Si la autenticación es exitosa, se emite un token JWT que incluye:
 - Identificador del usuario.
 - Rol asignado.
 - Tiempo de expiración del token (ej. 1 hora).
4. El token es enviado al cliente, que debe incluirlo en el encabezado `Authorization` con el esquema `Bearer` en cada solicitud protegida.

Middleware de Verificación

Se implementó un middleware genérico que intercepta todas las rutas protegidas para:

- Verificar la existencia del token JWT.
- Validar la firma y expiración del token.
- Extraer la identidad del usuario y su rol desde los claims del JWT.
- Pasar esta información al objeto `req.user` para ser utilizada por la lógica de autorización.

Control de Acceso Basado en Roles (RBAC)

Se definieron tres niveles de acceso:

- **Empleado (EMPLOYEE):**
 - Puede acceder exclusivamente a su propio perfil e información.
 - No puede acceder a los registros de otros usuarios.
- **Gerente (MANAGER):**
 - Puede acceder al perfil de los empleados bajo su cargo.
 - No tiene acceso a empleados de otras áreas ni a información de otros gerentes o administradores.
- **Administrador (ADMIN):**
 - Acceso completo a todos los recursos y operaciones del sistema.
 - Puede gestionar usuarios, editar roles y consultar registros globales.

Cada ruta protegida utiliza un middleware de autorización adicional que comprueba si el `req.user.role` tiene los privilegios necesarios para acceder al recurso solicitado.

Definición de Rutas y Protección

- **POST /auth/login:** Generación del token JWT.
- **GET /users/me:** Requiere autenticación; retorna el perfil del usuario autenticado.
- **GET /employees/:id:** Protegida; accesible solo si:
 - El solicitante es el propietario del perfil (EMPLOYEE) o
 - El solicitante es un MANAGER del empleado o
 - El solicitante es ADMIN.
- **GET /admin/users:** Solo para rol ADMIN.

Los accesos indebidos retornan un error **403 Forbidden** y se registran en los logs.

Casos de Prueba y Verificación

Se realizaron pruebas con Postman que verifican los siguientes escenarios:

- Acceso denegado al intentar usar el sistema sin token.
- Emisión correcta del token al autenticarse con credenciales válidas.
- Verificación de firma y expiración del token.
- Restricción de accesos a rutas según el rol:
 - Empleado accediendo a su perfil: ✓
 - Empleado accediendo a otro empleado: ✗
 - Gerente accediendo a subordinado: ✓
 - Gerente accediendo a otro gerente: ✗
 - Administrador accediendo a todos los recursos: ✓

Se confirmaron las respuestas esperadas (200, 403, 401) y el cumplimiento del flujo seguro.

Seguridad Adicional

- Hashing de contraseñas con salting (`bcrypt` con `saltRounds = 10`).
- Tokens firmados con clave secreta segura gestionada por variables de entorno.
- Expiración de tokens configurada por política (1 hora).
- Manejo de errores centralizado para evitar fugas de información.
- Validación de parámetros y entrada en todos los endpoints.

Consideraciones y Extensiones

El sistema está preparado para escalar e incluir funcionalidades adicionales como:

- Renovación de tokens mediante refresh tokens.

- Autenticación multifactor.
- Logs de auditoría y trazabilidad de acciones por usuario.
- Panel administrativo para la gestión de roles y usuarios.

Evidencias Técnicas

Se incluyen capturas de:

- Solicitud exitosa de login con generación del JWT.
- Intento de acceso a ruta protegida sin token (401).
- Acceso denegado por falta de permisos (403).
- Acceso correcto según jerarquía de roles.

Estructura del Repositorio

```
|— src/
|  |— controllers/
|  |— middleware/
|  |— models/
|  |— routes/
|  └─ utils/
|— .env
|— server.js
|— package.json
└─ README.md
```

Este sistema cumple con los principios de autenticación segura, autorización precisa y control de acceso basado en roles definidos, garantizando una segmentación clara de privilegios y una protección efectiva de los recursos internos. La implementación es compatible con arquitecturas modernas basadas en microservicios y se alinea con buenas prácticas de desarrollo seguro.

